

Experiment No.	3
Experiment Title	Regression Analysis using Linear and Regularized Models ¹
Student Name	Sanghamithra Namboodiri
Register Number	3122247001055
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	3rd August 2026

1 Aim and Objective

Aim: To implement and compare Linear Regression, Ridge, Lasso, and Elastic Net regression models for predicting the sanctioned loan amount from applicant and loan-related features, and to study how regularization influences model performance and generalization.

Objective: The aim of this experiment is to implement linear and regularized regression models – Linear Regression, Ridge, Lasso, and Elastic Net – to predict the sanctioned loan amount from applicant and loan-related features, and to study the effect of regularization strength on model performance and generalization. Specifically, the experiment evaluates:

- Baseline (unregularized) Linear Regression versus the three regularized variants.
- The effect of regularization strength (α) and, for Elastic Net, the L1/L2 mixing ratio.
- Generalization behaviour via 5-fold cross-validation.
- The effect of regularization on coefficient magnitudes.

2 Problem Statement

Given applicant demographic, financial, and property-related information, the task is to predict the continuous target variable **Loan Sanction Amount (USD)** – i.e. the amount of loan actually sanctioned to an applicant. The input consists of a mix of numerical features (income, credit score, property price, current loan expenses, etc.) and categorical features (profession, type of employment, location, gender, etc.). The output is a single continuous value, the sanctioned loan amount. The objective is to build and compare Linear, Ridge, Lasso, and Elastic Net regression models, tune their regularization hyperparameters, and identify the model that generalizes best while understanding the underlying bias–variance behaviour of the problem.

3 Dataset Description

Dataset Name	Predict Loan Amount Data
Dataset Source	Kaggle
Number of Samples	30,000 (original <code>train.csv</code>); 15,050 after duplicate removal; 13,520 after outlier removal (final training set used for modelling)

Number of Features	21 total columns (12 numeric, 9 categorical, including target); Customer ID , Name , and Property ID dropped as identifiers; expands to 38 features after one-hot encoding of categorical variables
Number of Classes	Not applicable – this is a regression problem with a continuous target (Loan Sanction Amount (USD))
Missing Values	Substantial missingness before cleaning: Type of Employment (7,270), Income (USD) (4,576), Property Age (4,850), Income Stability (1,683), out of 30,000 rows. -999 was also used as a sentinel for missing values in the target column and Property Price . All missingness was resolved to zero after preprocessing (see Section 5).
Train-Test Split	5-fold cross-validation used for hyperparameter tuning and model evaluation on the full 13,520-row training set; an additional 80/20 split held out separately to plot the training-vs-validation error curve; test.csv is a separate, unlabeled Kaggle competition test set used only for generating predictions.

4 Exploratory Data Analysis

Include all relevant visualizations.

- Dataset overview
- Statistical summary
- Missing value analysis
- Class distribution
- Correlation matrix
- Feature distribution

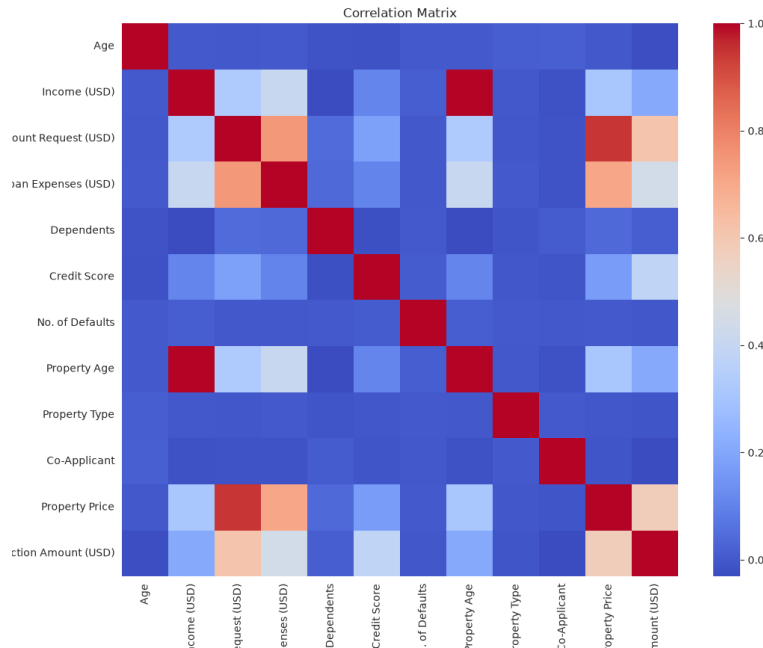


Figure 1: Correlation matrix of numeric features.

Inference: Loan Amount Request (USD), Current Loan Expenses (USD), and Property Price form a tightly correlated cluster (correlations in the 0.6–0.8 range), which is expected since larger requested loans tend to come with larger property values and expenses. Income (USD) and Property Age also show a strong pairwise correlation. Most other numeric features – Age, Dependents, Credit Score, No. of Defaults, Property Type, and Co-Applicant – show weak correlation with each other and with the target, Loan Sanction Amount (USD), indicating that the raw linear relationships in the data are concentrated in only a few features and that most of the signal for the target must come from a small subset of predictors plus the categorical variables.

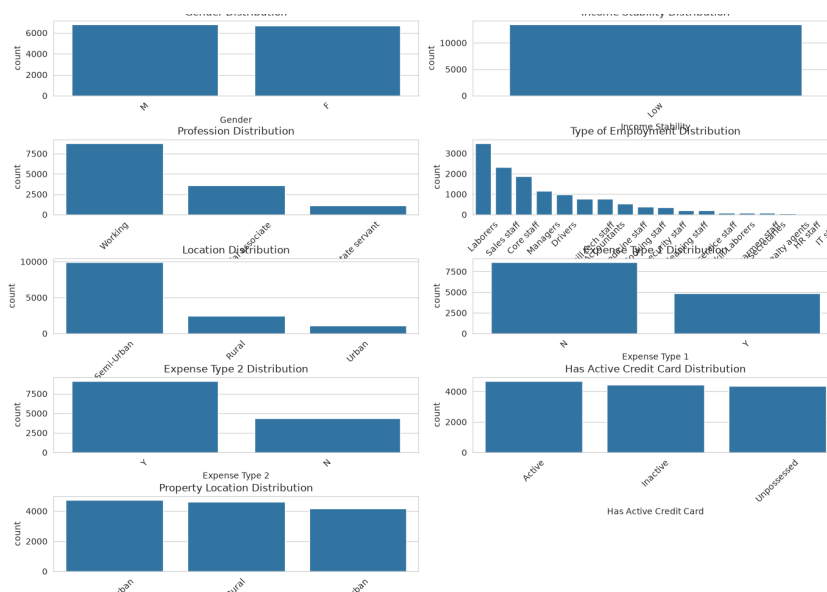


Figure 2: Distribution of categorical features.

Inference: Gender is fairly balanced between Male and Female applicants. **Income Stability** is heavily imbalanced, with almost all applicants recorded as "Low" after imputation. **Profession** is dominated by "Working" applicants (over 8,000), followed by "State servant" and "Commercial associate". **Type of Employment** is dominated by "Laborers", "Sales staff", and "Core staff", with many other employment types appearing only rarely – a long-tail distribution that explains why most of the one-hot encoded employment-type coefficients in the modelling results are small. **Location** is skewed toward "Semi-Urban", while **Property Location** is more evenly split across Semi-Urban, Rural, and Urban. **Expense Type 1** is mostly "N" while **Expense Type 2** is mostly "Y", and **Has Active Credit Card** is roughly evenly split across Active, Inactive, and Unpossessed.

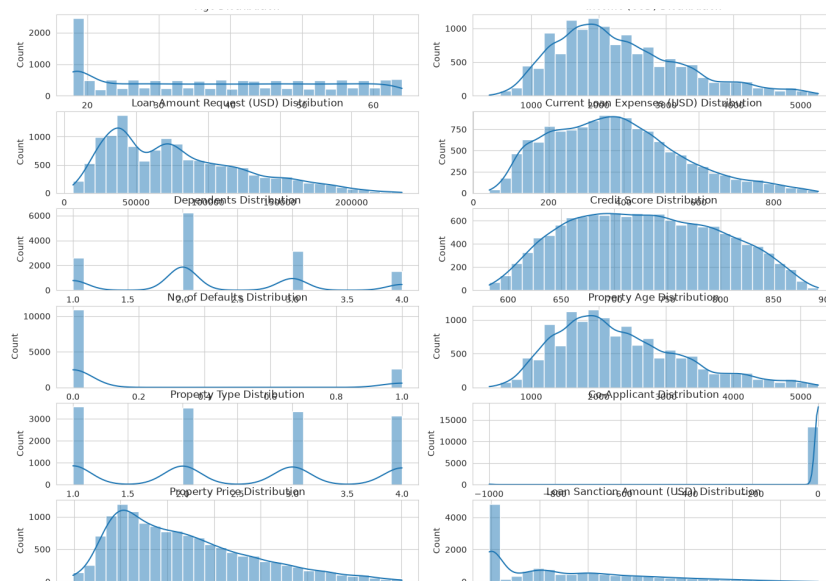


Figure 3: Distribution of numeric features.

Inference: Most numeric features are right-skewed: **Income (USD)**, **Loan Amount Request (USD)**, **Current Loan Expenses (USD)**, and **Co-Applicant** all show a long tail toward higher values. **Credit Score** and **Property Age** are closer to a broad, roughly bell-shaped distribution. Several features – **No. of Defaults**, **Property Type**, and **Property Price** – are effectively discrete/near-categorical, showing distinct spikes rather than a smooth continuous spread. Most strikingly, **Loan Sanction Amount (USD)**, the target variable, shows a massive spike at or near \$0 alongside a much smaller, broader spread across the rest of its range – direct visual confirmation of the bimodal, two-population target structure (a large "effectively rejected" group versus a smaller group sanctioned roughly proportionally to their request) that later limits every linear model's fit quality.

5 Data Preprocessing

1. **Identifier removal:** Customer ID, Name, and Property ID were dropped before modelling, as they carry no predictive signal.
2. **Missing value handling (train):** numeric columns were imputed with the training-

set median and categorical columns with the training-set mode, reducing missingness to zero across all columns.

3. **Missing value handling (test):** `test.csv` uses the placeholder string "?" for missing values in several columns. These were replaced with NaN, the affected columns coerced back to numeric dtype, and the same train-fitted median/mode imputers applied.
4. **Sentinel-value cleaning:** -999 values in **Property Price** and the target column were converted to NaN; rows with a missing target or missing **Property Price** were dropped from the training set (imputing the prediction target would leak information).
5. **Duplicate removal:** 14,950 duplicate rows were removed, reducing the training set from 30,000 to 15,050 rows.
6. **Outlier removal (IQR method):** for every numeric feature, values outside $[Q1 - 1.5 \text{ IQR}, Q3 + 1.5 \text{ IQR}]$ were removed (**No. of Defaults** and **Co-Applicant** were skipped due to near-zero IQR). This reduced the training set from 15,050 to 13,520 rows.
7. **Categorical encoding:** all categorical columns were one-hot encoded (`drop_first=True`); train and test were column-aligned so both share the exact same one-hot feature set.
8. **Feature scaling:** all features were standardized with `StandardScaler`, fit once on the full training set and reused for the test set, since Ridge/Lasso/Elastic Net penalize coefficient magnitude directly and are scale-sensitive.
9. **Train-test split:** an additional 80/20 split of the training data was held out purely to plot the training-vs-validation error curve; all cross-validation and final-model results use the full 13,520-row training set.
10. **Feature engineering:** not performed in this experiment; identified as a future-work item (e.g. an income-to-loan-request ratio feature).

6 Machine Learning Algorithm

Linear Regression

Working principle: fits a hyperplane to minimize the sum of squared residuals between predicted and actual values, with no penalty on coefficient magnitude.

Mathematical formulation: minimizes $\sum_i (y_i - \hat{y}_i)^2$ where $\hat{y}_i = \beta_0 + \sum_j \beta_j x_{ij}$.

Advantages: simple, interpretable, closed-form solution, no hyperparameters to tune.

Limitations: sensitive to multicollinearity and outliers; can overfit with many correlated features; cannot represent non-linear relationships or mixture-structured targets.

Ridge Regression

Working principle: adds an ℓ_2 penalty on coefficient magnitude to shrink coefficients smoothly toward zero without eliminating any of them.

Mathematical formulation: minimizes $\sum_i (y_i - \hat{y}_i)^2 + \alpha \sum_j \beta_j^2$.

Advantages: reduces variance and handles multicollinearity well; has a closed-form solution and is fast to train.

Limitations: does not perform feature selection (all coefficients remain non-zero); an inappropriate α can under- or over-shrink coefficients.

Lasso Regression

Working principle: adds an ℓ_1 penalty on coefficient magnitude, which can drive some coefficients exactly to zero, performing implicit feature selection.

Mathematical formulation: minimizes $\sum_i (y_i - \hat{y}_i)^2 + \alpha \sum_j |\beta_j|$.

Advantages: produces sparser, more interpretable models; useful when only a subset of features is expected to be relevant.

Limitations: solved iteratively via coordinate descent (slower than Ridge); can behave erratically with strongly correlated features, arbitrarily selecting one over another.

Elastic Net

Working principle: combines the ℓ_1 and ℓ_2 penalties, balancing sparsity (Lasso) with the smooth shrinkage and multicollinearity handling of Ridge.

Mathematical formulation: minimizes $\sum_i (y_i - \hat{y}_i)^2 + \alpha \left(r \sum_j |\beta_j| + \frac{1-r}{2} \sum_j \beta_j^2 \right)$, where r is the ℓ_1 mixing ratio.

Advantages: more robust than Lasso alone when features are highly correlated; two hyperparameters (α, r) give finer control over the bias-variance trade-off.

Limitations: two hyperparameters to tune (larger search space); slowest of the four models to train due to balancing both penalty terms simultaneously.

7 Experimental Setup

Python Version
Scikit-Learn Version
IDE
Operating System
Hardware

8 Hyperparameter Selection

Grid Search with 5-fold cross-validation (`GridSearchCV`, `scoring = R2`) was used for Ridge, Lasso, and Elastic Net. Linear Regression has no hyperparameters and was fit directly as the baseline.

Parameter	Value
Ridge α search space	{0.01, 0.1, 1, 10, 100}
Lasso α search space	{0.001, 0.01, 0.1, 1, 10}
Elastic Net α search space	{0.01, 0.1, 1, 10}
Elastic Net ℓ_1 ratio search space	{0.2, 0.5, 0.8}
Ridge – Best α	10 (Best CV $R^2 = 0.4595$)
Lasso – Best α	10 (Best CV $R^2 = 0.4596$)
Elastic Net – Best α, ℓ_1 ratio	0.01, 0.8 (Best CV $R^2 = 0.4595$)

9 Implementation Details

- **Language and core libraries:** Python, using `pandas` and `numpy` for data handling, `matplotlib/seaborn` for visualization, and `scikit-learn` for modelling (`LinearRegression`, `Ridge`, `Lasso`, `ElasticNet`, `GridSearchCV`, `StandardScaler`, `OneHotEncoder`/`pd.get_dummies`).
- **Pipeline structure:** raw `train.csv` and `test.csv` are loaded, cleaned (identifier removal, missing-value imputation, sentinel-value handling, duplicate and outlier removal), one-hot encoded, column-aligned between train and test, and standardized with a single `StandardScaler` fit on the training set.
- **Model training:** each of the four models (`Linear Regression`, `Ridge`, `Lasso`, `Elastic Net`) is instantiated with `scikit-learn` defaults except for the tuned hyperparameters found via `GridSearchCV`; all models are trained on the same standardized, encoded feature matrix so that results are directly comparable.
- **Cross-validation:** `GridSearchCV` with `cv=5` and `scoring='r2'` is used to select α (and, for Elastic Net, the ℓ_1 ratio) for Ridge, Lasso, and Elastic Net; the same 5-fold split is used to report MAE, MSE, RMSE, and R^2 via `cross_val_predict/cross_val_score` for all four models, including the untuned Linear Regression baseline.
- **Learning curve:** an additional, separate 80/20 train-validation split (independent of the 5-fold CV used for tuning and metric reporting) is used purely to plot training vs. validation MSE for each model, in order to diagnose overfitting vs. underfitting.
- **Coefficient extraction:** the `.coef_` attribute is extracted from each fitted model on the standardized feature set, enabling a direct, scale-comparable comparison of coefficient magnitudes across all four models (Table 2).
- **Timing:** wall-clock training time for each final model (fit once on the 80% train split) is measured with Python's `time` module.
- **Test-set inference:** all four fitted models generate predictions on the cleaned, encoded, and scaled `test.csv`; since `test.csv` has no ground-truth target, these predictions are exported but not scored.

10 Results

As this is a regression task, standard classification metrics (Accuracy, Precision, Recall, F1, ROC-AUC) do not apply; results are reported using regression metrics on the 5-fold cross-validation of the full training set ($n = 13,520$).

Model	MAE	MSE	RMSE	R ²
Linear Regression	22,251.52	8.4946×10^8	29,145.56	0.4595
Ridge Regression	22,253.66	8.4946×10^8	29,145.48	0.4595
Lasso Regression	22,250.21	8.4931×10^8	29,142.92	0.4596
Elastic Net Regression	22,256.10	8.4947×10^8	29,145.60	0.4595

Training time (final models, fit on 80% train split): Ridge – 0.0060s; Linear Regression – 0.1084s; Lasso Regression – 0.4975s; Elastic Net – 2.1238s.

Test set performance: not available. `test.csv` is an unlabeled Kaggle competition test set with no target column, so held-out ground-truth metrics could not be computed; predictions were nonetheless generated for every row using all four fitted models.

11 Result Visualization

Include all graphs generated during the experiment. Recommended figures for this regression task (adapted from the standard classification set): target/class distribution, correlation/feature scatter, predicted-vs-actual plot, residual plot, training-vs-validation error (learning curve), and coefficient (feature importance) comparison.

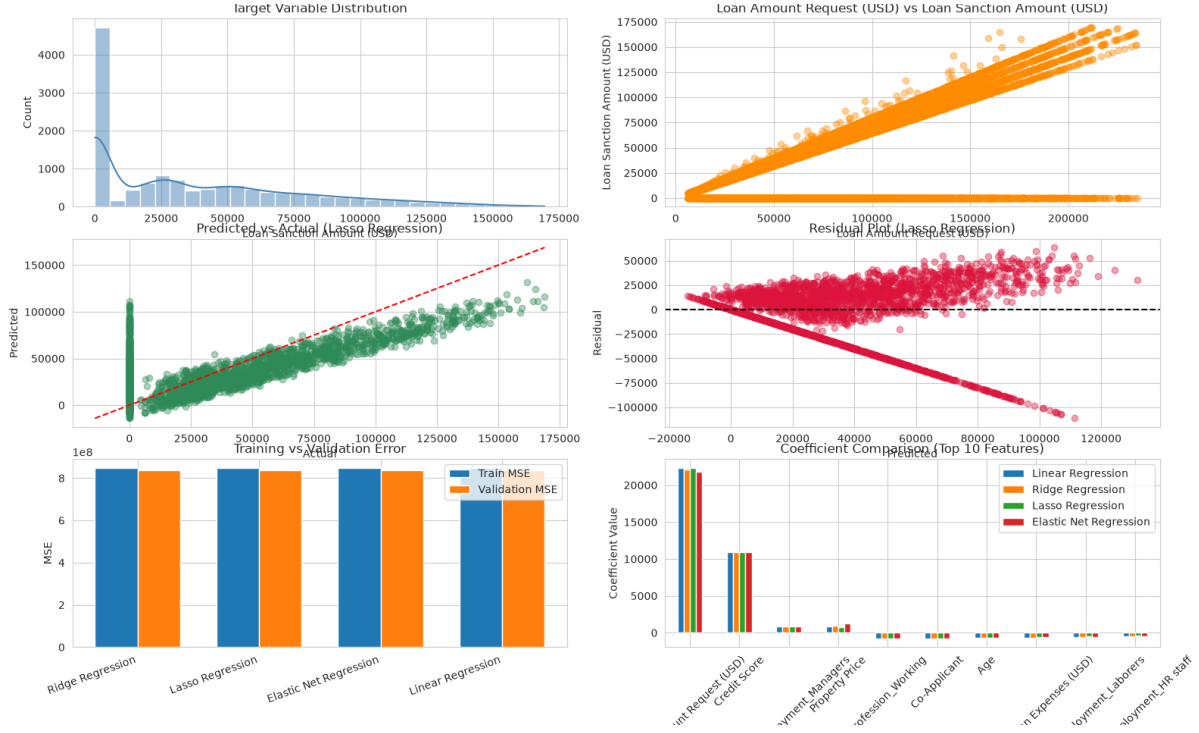


Figure 4: Combined regression results dashboard: (c) predicted vs. actual sanctioned amount for the best model (Lasso Regression), (d) residual plot for the same model, (e) training vs. validation error across all four models, and (f) coefficient comparison across models (top 10 features).

Inference: Panel (c) plots predictions against the $y = x$ reference line. A vertical cluster of points sits at Actual $\approx$ \$0 but is predicted anywhere from about $-\$20,000$ to over $\$100,000$, showing the model cannot identify which "near-zero" applications belong to the rejected population; for the proportional population, points form a diagonal band that runs consistently below the $y = x$ line at higher amounts, i.e. the model systematically underpredicts large sanctioned amounts. Panel (d)'s residuals confirm this: rather than random scatter around zero, they form two distinct diagonal bands – one trending up to roughly $+\$50,000$ as predicted value increases, and one trending down to about $-\$100,000$ – the residual-space signature of the same rejected-vs-proportional mixture. Panel (e) shows train MSE ($\approx 8.5 \times 10^8$) and validation MSE ($\approx 8.3 \times 10^8$) are nearly identical for all four models, ruling out overfitting and confirming all four are hitting the same underfitting ceiling caused by the bimodal target structure. Panel (f) shows Loan Amount Request (USD) ($\sim 22,000$) and Credit Score ($\sim 10,900$) dominate by more than an order of magnitude over every other feature, including Type of Employment Managers, Property Price, Profession Working, Co-Applicant, Age, and Current Loan Expenses (USD). A full feature-by-feature coefficient comparison across all 38 features and all four models is given in Table 2 below.

Table 2: Coefficient comparison across all four models (standardized features)

Feature	Linear	Ridge	Lasso	Elastic Net
Age	-683.85	-683.53	-674.55	-682.10

Feature	Linear	Ridge	Lasso	Elastic Net
Income (USD)	−225.55	−224.35	−21.63	−222.27
Loan Amount Request (USD)	22,358.89	22,144.93	22,336.28	21,806.00
Current Loan Expenses (USD)	−661.75	−634.96	−626.84	−591.72
Dependents	−351.08	−350.05	−340.49	−348.39
Credit Score	10,940.37	10,935.23	10,941.57	10,926.55
No. of Defaults	−312.64	−313.05	−302.76	−313.66
Property Age	−225.55	−224.35	−415.98	−222.53
Property Type	−200.48	−200.51	−190.80	−200.54
Co-Applicant	−755.22	−754.92	−745.45	−754.41
Property Price	811.24	996.24	795.61	1,288.10
Gender_M	−65.78	−65.04	−55.47	−63.87
Profession_State servant	−294.35	−293.13	−270.11	−291.12
Profession_Working	−785.62	−784.52	−767.79	−782.72
Type of Employment_Cleaning staff	−265.71	−265.73	−234.59	−265.66
Type of Employment_Cooking staff	−113.33	−114.45	−78.11	−116.17
Type of Employment_Core staff	−37.73	−37.75	3.10	−37.63
Type of Employment_Drivers	−340.57	−342.89	−297.19	−346.56
Type of Employment_HR staff	−429.87	−429.51	−411.63	−428.89
Type of Employment_High skill tech	−171.81	−171.84	−127.42	−171.75
Type of Employment_IT staff	130.64	130.80	127.23	131.06
Type of Employment_Laborers	−596.74	−598.64	−524.00	−601.56
Type of Employment_Low-skill Laborers	12.10	11.33	14.54	10.08
Type of Employment_Managers	817.63	816.35	847.40	814.40
Type of Employment_Medicine staff	101.77	101.06	120.05	100.02
Type of Employment_Private service	−354.08	−354.32	−325.20	−354.64
Type of Employment_Realty agents	−88.81	−88.57	−68.92	−88.16
Type of Employment_Sales staff	−140.52	−142.52	−71.81	−145.58
Type of Employment_Secretaries	182.94	182.82	184.59	182.64
Type of Employment_Security staff	−159.02	−158.80	−124.44	−158.38
Type of Employment_Waiters/barmen staff	385.91	384.79	388.89	382.97
Location_Semi-Urban	33.19	33.33	27.74	33.60
Location_Urban	−306.71	−305.22	−300.46	−302.78
Expense Type 1_Y	−346.90	−348.06	−340.15	−349.89
Expense Type 2_Y	393.84	392.94	381.43	391.49
Has Active Credit Card_Inactive	356.48	356.75	337.21	357.15
Has Active Credit Card_Unpossessed	317.35	317.03	298.62	316.47
Property Location_Semi-Urban	−146.71	−146.11	−139.40	−145.13
Property Location_Urban	109.58	110.08	103.34	110.89

12 Overfitting and Underfitting Analysis

Cross-validation performance is essentially flat across all four models: Linear, Ridge, Lasso, and Elastic Net all land within $0.0001 R^2$ of each other (0.4595–0.4596), and their coefficient vectors are nearly identical in both sign and magnitude. This strongly suggests regularization is not the limiting factor – an ℓ_2 , ℓ_1 , or combined penalty barely moves the

model away from the unregularized OLS solution.

Grid search selected $\alpha = 10$ for both Ridge and Lasso (mid-range of their search grids) and a very small $\alpha = 0.01$ for Elastic Net, meaning only light shrinkage was preferred in every case. If overfitting were the main problem, stronger regularization would be expected to noticeably outperform the baseline – it does not. The only feature Lasso drove meaningfully toward zero was **Income (USD)** (from -225.5 to -21.6) and **Type of Employment_Core staff** (from -37.7 to $+3.1$); every other coefficient survives largely intact.

This pattern points to underfitting, not overfitting. An R^2 of ≈ 0.46 means the entire linear family explains under half of the variance in sanctioned loan amount, and the target distribution is bimodal: loans sanctioned near \$0 alongside loans sanctioned roughly proportionally to the requested amount. The residual plot shows the direct consequence – residuals split into two clean diagonal bands rather than scattering randomly around zero, the textbook signature of a single linear model trying to fit two structurally different subpopulations at once. Train and validation MSE are essentially equal for every model, confirming this is not overfitting to noise but a limitation of the model architecture itself.

13 Bias–Variance Analysis

All four models converge to nearly the same coefficients and the same CV R^2 regardless of regularization strength – a high-bias, low-variance regime. Ridge’s smooth ℓ_2 shrinkage barely perturbs the OLS coefficients (e.g. **Loan Amount Request (USD)**: $22,358.89 \rightarrow 22,144.93$), confirming there was little variance for it to reduce. Lasso only meaningfully shrank two of 38 features toward zero and marginally improved CV R^2 (0.4596 vs. 0.4595). Elastic Net, leaning mostly Lasso-like at ℓ_1 ratio = 0.8 but with a much smaller $\alpha = 0.01$, sits essentially on top of the other three models.

The dominant issue is bias, not variance: two features (**Loan Amount Request (USD)** and **Credit Score**) carry coefficients an order of magnitude larger than everything else, and the model still only reaches $R^2 \approx 0.46$. The target variable is a mixture of two populations that a single linear equation cannot represent at once – it compromises between them, systematically underpredicting the proportional group and overpredicting the near-zero group. No amount of ℓ_1/ℓ_2 regularization on this feature set can close that gap, since the problem is the functional form, not coefficient magnitude.

14 Observations and Conclusion

Observations

- All four models – Linear, Ridge, Lasso, and Elastic Net – converge to nearly identical CV R^2 scores (0.4595–0.4596) and nearly identical coefficient vectors, indicating regularization has almost no effect on this dataset/feature combination.
- Grid search consistently prefers light regularization ($\alpha = 10$ for Ridge/Lasso out of grids extending to 100; $\alpha = 0.01$ for Elastic Net), reinforcing that the unregularized OLS solution is already close to optimal for this feature set.
- **Loan Amount Request (USD)** and **Credit Score** dominate every model’s coefficients by more than an order of magnitude over all other features.

- The predicted-vs-actual and residual plots both show a clear two-band structure, directly traceable to the bimodal (rejected vs. proportionally-sanctioned) nature of the target variable.
- Train and validation MSE are nearly identical across all models, ruling out overfitting and confirming the $R^2 \approx 0.46$ ceiling is a bias (underfitting) problem rather than a variance problem.
- Lasso achieves the best (marginal) CV R^2 and is also the only model to perform any meaningful feature shrinkage, zeroing in on `Income (USD)` and `Type of Employment_Core staff` as the least useful predictors.

Conclusion

- **Best-performing model (by CV R^2):** Lasso Regression ($\alpha = 10$) marginally leads with $R^2 = 0.4596$ and the lowest MAE/RMSE among the four models, though the margin over the others is negligible ($< 0.0002 R^2$).
- **Regularization had minimal impact:** all four models converge to nearly identical coefficients and CV scores, confirming the model is not overfitting the already outlier-filtered 13,520-row training set.
- **Dominant predictors:** `Loan Amount Request (USD)` and `Credit Score` dwarf every other coefficient by more than an order of magnitude.
- **Bias–variance trade-off:** near-identical performance across regularization strengths points to a high-bias, low-variance regime – the linear model family itself, not overfitting, caps performance around $R^2 \approx 0.46$.
- **Training time:** Ridge trains near-instantly (closed-form solution); Lasso and Elastic Net are slower due to iterative coordinate descent, though all four remain practically fast at this dataset size.
- **Overall recommendation:** given the negligible difference between models, Ridge Regression is the most practical choice – fastest to train, statistically indistinguishable performance from Lasso/Elastic Net, and unlike Lasso it does not risk zeroing out a feature that could matter on unseen data.
- **Limitations and future work:** an R^2 around 0.46 leaves more than half the variance unexplained, driven by the target’s mixture structure (rejected vs. approved-proportionally loans). Future directions include a two-stage model (classify approval, then regress amount for approved cases), engineering interaction features (e.g. income-to-loan-request ratio), trying non-linear models (gradient boosting, random forest), and obtaining true labels for `test.csv` to measure genuine held-out generalization.

15 References

1. Kaggle, "Predict Loan Amount Data," [Online]. Available: <https://www.kaggle.com/>.

2. F. Pedregosa *et al.*, "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
3. T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning*, 2nd ed. New York: Springer, 2009.